第二部分

结构化并发应用程序

第6章

任务执行

大多数并发应用程序都是围绕“任务执行（Task Execution）”来构造的：任务通常是一些抽象的且离散的工作单元。通过把应用程序的工作分解到多个任务中，可以简化程序的组织结构，提供一种自然的事务边界来优化错误恢复过程，以及提供一种自然的并行工作结构来提升并发性。

6.1 在线程中执行任务

当围绕“任务执行”来设计应用程序结构时，第一步就是要找出清晰的任务边界。在理想情况下，各个任务之间是相互独立的：任务并不依赖于其他任务的状态、结果或边界效应。独立性有助于实现并发，因为如果存在足够多的处理资源，那么这些独立的任务都可以并行执行。为了在调度与负载均衡等过程中实现更高的灵活性，每项任务还应该表示应用程序的一小部分处理能力。

在正常的负载下，服务器应用程序应该同时表现出良好的吞吐量和快速的响应性。应用程序提供商希望程序支持尽可能多的用户，从而降低每个用户的服务成本，而用户则希望获得尽快的响应。而且，当负荷过载时，应用程序的性能应该是逐渐降低，而不是直接失败。要实现上述目标，应该选择清晰的任务边界以及明确的任务执行策略（请参见 6.2.2 节）。

大多数服务器应用程序都提供了一种自然的任务边界选择方式：以独立的客户请求为边界。Web 服务器、邮件服务器、文件服务器、EJB 容器以及数据库服务器等，这些服务器都能通过网络接受远程客户的连接请求。将独立的请求作为任务边界，既可以实现任务的独立性，又可

以实现合理的任务规模。例如，在向邮件服务器提交一个消息后得到的结果，并不会受其他正在处理的消息影响，而且在处理单个消息时通常只需要服务器总处理能力的很小一部分。

6.1.1　串行地执行任务

在应用程序中可以通过多种策略来调度任务，而其中一些策略能够更好地利用潜在的并发性。最简单的策略就是在单个线程中串行地执行各项任务。程序清单 6-1 中的 SingleThreadWebServer 将串行地处理它的任务（即通过 80 端口接收到的 HTTP 请求）。至于如何处理请求的细节问题，在这里并不重要，我们感兴趣的是如何表征不同调度策略的同步特性。

程序清单 6-1　串行的 Web 服务器

```
class SingleThreadWebServer {
    public static void main(String[] args) throws IOException {
        ServerSocket socket = new ServerSocket(80);
        while (true) {
            Socket connection = socket.accept();
            handleRequest(connection);
        }
    }
}
```

SingleThreadWebServer 很简单，且在理论上是正确的，但在实际生产环境中的执行性能却很糟糕，因为它每次只能处理一个请求。主线程在接受连接与处理相关请求等操作之间不断地交替运行。当服务器正在处理请求时，新到来的连接必须等待直到请求处理完成，然后服务器将再次调用 accept。如果处理请求的速度很快并且 handleRequest 可以立即返回，那么这种方法是可行的，但现实世界中的 Web 服务器的情况却并非如此。

在 Web 请求的处理中包含了一组不同的运算与 I/O 操作。服务器必须处理套接字 I/O 以读取请求和写回响应，这些操作通常会由于网络拥塞或连通性问题而被阻塞。此外，服务器还可能处理文件 I/O 或者数据库请求，这些操作同样会阻塞。在单线程的服务器中，阻塞不仅会推迟当前请求的完成时间，而且还将彻底阻止等待中的请求被处理。如果请求阻塞的时间过长，用户将认为服务器是不可用的，因为服务器看似失去了响应。同时，服务器的资源利用率非常低，因为当单线程在等待 I/O 操作完成时，CPU 将处于空闲状态。

在服务器应用程序中，串行处理机制通常都无法提供高吞吐率或快速响应性。也有一些例外，例如，当任务数量很少且执行时间很长时，或者当服务器只为单个用户提供服务，并且该客户每次只发出一个请求时——但大多数服务器应用程序并不是按照这种方式来工作的[⊖]。

6.1.2　显式地为任务创建线程

通过为每个请求创建一个新的线程来提供服务，从而实现更高的响应性，如程序清单 6-2 中的 ThreadPerTaskWebServer 所示。

⊖ 在某些情况中，串行处理方式能带来简单性或安全性。大多数 GUI 框架都通过单一的线程来串行地处理任务。我们将在第 9 章再次介绍串行模型。

程序清单 6-2 在 Web 服务器中为每个请求启动一个新的线程（不要这么做）

```
class ThreadPerTaskWebServer {
    public static void main(String[] args) throws IOException {
        ServerSocket socket = new ServerSocket(80);
        while (true) {
            final  Socket connection = socket.accept();
            Runnable task = new Runnable() {
                    public void run() {
                        handleRequest(connection);
                    }
                };
            new Thread(task).start();
        }
    }
}
```

ThreadPerTaskWebServer 在结构上类似于前面的单线程版本——主线程仍然不断地交替执行“接受外部连接”与“分发请求”等操作。区别在于，对于每个连接，主循环都将创建一个新线程来处理请求，而不是在主循环中进行处理。由此可得出 3 个主要结论：

- 任务处理过程从主线程中分离出来，使得主循环能够更快地重新等待下一个到来的连接。这使得程序在完成前面的请求之前可以接受新的请求，从而提高响应性。
- 任务可以并行处理，从而能同时服务多个请求。如果有多个处理器，或者任务由于某种原因被阻塞，例如等待 I/O 完成、获取锁或者资源可用性等，程序的吞吐量将得到提高。
- 任务处理代码必须是线程安全的，因为当有多个任务时会并发地调用这段代码。

在正常负载情况下，“为每个任务分配一个线程”的方法能提升串行执行的性能。只要请求的到达速率不超出服务器的请求处理能力，那么这种方法可以同时带来更快的响应性和更高的吞吐率。

6.1.3 无限制创建线程的不足

在生产环境中，“为每个任务分配一个线程”这种方法存在一些缺陷，尤其是当需要创建大量的线程时：

线程生命周期的开销非常高。线程的创建与销毁并不是没有代价的。根据平台的不同，实际的开销也有所不同，但线程的创建过程都会需要时间，延迟处理的请求，并且需要 JVM 和操作系统提供一些辅助操作。如果请求的到达率非常高且请求的处理过程是轻量级的，例如大多数服务器应用程序就是这种情况，那么为每个请求创建一个新线程将消耗大量的计算资源。

资源消耗。活跃的线程会消耗系统资源，尤其是内存。如果可运行的线程数量多于可用处理器的数量，那么有些线程将闲置。大量空闲的线程会占用许多内存，给垃圾回收器带来压力，而且大量线程在竞争 CPU 资源时还将产生其他的性能开销。如果你已经拥有足够多的线程使所有 CPU 保持忙碌状态，那么再创建更多的线程反而会降低性能。

稳定性。在可创建线程的数量上存在一个限制。这个限制值将随着平台的不同而不同，并且受多个因素制约，包括 JVM 的启动参数、Thread 构造函数中请求的栈大小，以及底层操作

系统对线程的限制等[⊖]。如果破坏了这些限制，那么很可能抛出 OutOfMemoryError 异常，要想从这种错误中恢复过来是非常危险的，更简单的办法是通过构造程序来避免超出这些限制。

在一定的范围内，增加线程可以提高系统的吞吐率，但如果超出了这个范围，再创建更多的线程只会降低程序的执行速度，并且如果过多地创建一个线程，那么整个应用程序将崩溃。要想避免这种危险，就应该对应用程序可以创建的线程数量进行限制，并且全面地测试应用程序，从而确保在线程数量达到限制时，程序也不会耗尽资源。

“为每个任务分配一个线程”这种方法的问题在于，它没有限制可创建线程的数量，只限制了远程用户提交 HTTP 请求的速率。与其他的并发危险一样，在原型设计和开发阶段，无限制地创建线程或许还能较好地运行，但在应用程序部署后并处于高负载下运行时，才会有问题不断地暴露出来。因此，某个恶意的用户或者过多的用户，都会使 Web 服务器的负载达到某个阈值，从而使服务器崩溃。如果服务器需要提供高可用性，并且在高负载情况下能平缓地降低性能，那么这将是一个严重的故障。

6.2 Executor 框架

任务是一组逻辑工作单元，而线程则是使任务异步执行的机制。我们已经分析了两种通过线程来执行任务的策略，即把所有任务放在单个线程中串行执行，以及将每个任务放在各自的线程中执行。这两种方式都存在一些严格的限制：串行执行的问题在于其糟糕的响应性和吞吐量，而“为每个任务分配一个线程”的问题在于资源管理的复杂性。

在第 5 章中，我们介绍了如何通过有界队列来防止高负荷的应用程序耗尽内存。线程池简化了线程的管理工作，并且 java.util.concurrent 提供了一种灵活的线程池实现作为 Executor 框架的一部分。在 Java 类库中，任务执行的主要抽象不是 Thread，而是 Executor，如程序清单 6-3 所示。

程序清单 6-3 Executor 接口

```
public interface Executor {
    void execute(Runnable command);
}
```

虽然 Executor 是个简单的接口，但它却为灵活且强大的异步任务执行框架提供了基础，该框架能支持多种不同类型的任务执行策略。它提供了一种标准的方法将任务的提交过程与执行过程解耦开来，并用 Runnable 来表示任务。Executor 的实现还提供了对生命周期的支持，以及统计信息收集、应用程序管理机制和性能监视等机制。

Executor 基于生产者－消费者模式，提交任务的操作相当于生产者（生成待完成的工作单元），执行任务的线程则相当于消费者（执行完这些工作单元）。如果要在程序中实现一个生产

⊖ 在 32 位的机器上，其中一个主要的限制因素是线程栈的地址空间。每个线程都维护两个执行栈，一个用于 Java 代码，另一个用于原生代码。通常，JVM 在默认情况下会生成一个复合的栈，大小约为 0.5MB。（可以通过 JVM 标志 -Xss 或者通过 Thread 的构造函数来修改这个值。）如果将 2^{32} 除以每个线程的栈大小，那么线程数量将被限制为几千到几万。其他一些因素，例如操作系统的限制等，则可能会施加更加严格的约束。

者－消费者的设计，那么最简单的方式通常就是使用 Executor。

6.2.1 示例：基于 Executor 的 Web 服务器

基于 Executor 来构建 Web 服务器是非常容易的。在程序清单 6-4 中用 Executor 代替了硬编码的线程创建过程。在这种情况下使用了一种标准的 Executor 实现，即一个固定长度的线程池，可以容纳 100 个线程。

程序清单 6-4 基于线程池的 Web 服务器

```
class TaskExecutionWebServer {
    private static final int NTHREADS = 100;
    private static final Executor exec
        = Executors.newFixedThreadPool(NTHREADS);

    public static void main(String[] args) throws IOException {
        ServerSocket socket = new ServerSocket(80);
        while (true) {
            final Socket connection = socket.accept();
            Runnable task = new Runnable() {
                public void run() {
                    handleRequest(connection);
                }
            };
            exec.execute(task);
        }
    }
}
```

在 TaskExecutionWebServer 中，通过使用 Executor，将请求处理任务的提交与任务的实际执行解耦开来，并且只需采用另一种不同的 Executor 实现，就可以改变服务器的行为。改变 Executor 实现或配置所带来的影响要远远小于改变任务提交方式带来的影响。通常，Executor 的配置是一次性的，因此在部署阶段可以完成，而提交任务的代码却会不断地扩散到整个程序中，增加了修改的难度。

我们可以很容易地将 TaskExecutionWebServer 修改为类似 ThreadPerTaskWebServer 的行为，只需使用一个为每个请求都创建新线程的 Executor。编写这样的 Executor 很简单，如程序清单 6-5 中的 ThreadPerTaskExecutor 所示。

程序清单 6-5 为每个请求启动一个新线程的 Executor

```
public class ThreadPerTaskExecutor implements Executor {
    public void execute(Runnable r) {
        new Thread(r).start();
    };
}
```

同样，还可以编写一个 Executor 使 TaskExecutionWebServer 的行为类似于单线程的行为，即以同步的方式执行每个任务，然后再返回，如程序清单 6-6 中的 WithinThreadExecutor 所示。

程序清单 6-6 在调用线程中以同步方式执行所有任务的 Executor

```
public class WithinThreadExecutor implements Executor {
    public void execute(Runnable r) {
        r.run();
    };
}
```

6.2.2 执行策略

通过将任务的提交与执行解耦开来，从而无须太大的困难就可以为某种类型的任务指定和修改执行策略。在执行策略中定义了任务执行的“What、Where、When、How”等方面，包括：

- 在什么（What）线程中执行任务？
- 任务按照什么（What）顺序执行（FIFO、LIFO、优先级）？
- 有多少个（How Many）任务能并发执行？
- 在队列中有多少个（How Many）任务在等待执行？
- 如果系统由于过载而需要拒绝一个任务，那么应该选择哪一个（Which）任务？另外，如何 (How) 通知应用程序有任务被拒绝？
- 在执行一个任务之前或之后，应该进行哪些（What）动作？

各种执行策略都是一种资源管理工具，最佳策略取决于可用的计算资源以及对服务质量的需求。通过限制并发任务的数量，可以确保应用程序不会由于资源耗尽而失败，或者由于在稀缺资源上发生竞争而严重影响性能[⊖]。通过将任务的提交与任务的执行策略分离开来，有助于在部署阶段选择与可用硬件资源最匹配的执行策略。

每当看到下面这种形式的代码时：

```
new Thread(runnable).start()
```

并且你希望获得一种更灵活的执行策略时，请考虑使用 Executor 来代替 Thread。

6.2.3 线程池

线程池，从字面含义来看，是指管理一组同构工作线程的资源池。线程池是与工作队列（Work Queue）密切相关的，其中在工作队列中保存了所有等待执行的任务。工作者线程（Worker Thread）的任务很简单：从工作队列中获取一个任务，执行任务，然后返回线程池并等待下一个任务。

“在线程池中执行任务”比“为每个任务分配一个线程”优势更多。通过重用现有的线程而不是创建新线程，可以在处理多个请求时分摊在线程创建和销毁过程中产生的巨大开销。另一个额外的好处是，当请求到达时，工作线程通常已经存在，因此不会由于等待创建线程而延

⊖ 这类似于某个企业应用程序中事务监视器 (Transaction Monitor) 的作用：它能将事务的执行速率控制在某个合理水平，因而就不会使有限资源耗尽或者造成过大压力。

迟任务的执行，从而提高了响应性。通过适当调整线程池的大小，可以创建足够多的线程以便使处理器保持忙碌状态，同时还可以防止过多线程相互竞争资源而使应用程序耗尽内存或失败。

类库提供了一个灵活的线程池以及一些有用的默认配置。可以通过调用 Executors 中的静态工厂方法之一来创建一个线程池：

newFixedThreadPool。newFixedThreadPool 将创建一个固定长度的线程池，每当提交一个任务时就创建一个线程，直到达到线程池的最大数量，这时线程池的规模将不再变化（如果某个线程由于发生了未预期的 Exception 而结束，那么线程池会补充一个新的线程）。

newCachedThreadPool。newCachedThreadPool 将创建一个可缓存的线程池，如果线程池的当前规模超过了处理需求时，那么将回收空闲的线程，而当需求增加时，则可以添加新的线程，线程池的规模不存在任何限制。

newSingleThreadExecutor。newSingleThreadExecutor 是一个单线程的 Executor，它创建单个工作者线程来执行任务，如果这个线程异常结束，会创建另一个线程来替代。newSingleThreadExecutor 能确保依照任务在队列中的顺序来串行执行（例如 FIFO、LIFO、优先级）。[⊖]

newScheduledThreadPool。newScheduledThreadPool 创建了一个固定长度的线程池，而且以延迟或定时的方式来执行任务，类似于 Timer（参见 6.2.5 节）。

newFixedThreadPool 和 newCachedThreadPool 这两个工厂方法返回通用的 ThreadPoolExecutor 实例，这些实例可以直接用来构造专门用途的 executor。我们将在第 8 章中深入讨论线程池的各个配置选项。

TaskExecutionWebServer 中的 Web 服务器使用了一个带有有界线程池的 Executor。通过 execute 方法将任务提交到工作队列中，工作线程反复地从工作队列中取出任务并执行它们。

从“为每任务分配一个线程”策略变成基于线程池的策略，将对应用程序的稳定性产生重大的影响：Web 服务器不会再在高负载情况下失败[⊖]。由于服务器不会创建数千个线程来争夺有限的 CPU 和内存资源，因此服务器的性能将平缓地降低。通过使用 Executor，可以实现各种调优、管理、监视、记录日志、错误报告和其他功能，如果不使用任务执行框架，那么要增加这些功能是非常困难的。

6.2.4 Executor 的生命周期

我们已经知道如何创建一个 Executor，但并没有讨论如何关闭它。Executor 的实现通常会创建线程来执行任务。但 JVM 只有在所有（非守护）线程全部终止后才会退出。因此，如果无法正确地关闭 Executor，那么 JVM 将无法结束。

⊖ 单线程的 Executor 还提供了大量的内部同步机制，从而确保了任务执行的任何内存写入操作对于后续任务来说都是可见的。这意味着，即使这个线程会不时地被另一个线程替代，但对象总是可以安全地封闭在“任务线程”中。

⊖ 尽管服务器不会因为创建了过多的线程而失败，但在足够长的时间内，如果任务到达的速度总是超过任务执行的速度，那么服务器仍有可能（只是更不易）耗尽内存，因为等待执行的 Runnable 队列将不断增长。可以通过使用一个有界工作队列在 Executor 框架内部解决这个问题（参见 8.3.2 节）。

由于 Executor 以异步方式来执行任务，因此在任何时刻，之前提交任务的状态不是立即可见的。有些任务可能已经完成，有些可能正在运行，而其他的任务可能在队列中等待执行。当关闭应用程序时，可能采用最平缓的关闭形式（完成所有已经启动的任务，并且不再接受任何新的任务），也可能采用最粗暴的关闭形式（直接关掉机房的电源），以及其他各种可能的形式。既然 Executor 是为应用程序提供服务的，因而它们也是可关闭的（无论采用平缓的方式还是粗暴的方式），并将在关闭操作中受影响的任务的状态反馈给应用程序。

为了解决执行服务的生命周期问题，Executor 扩展了 ExecutorService 接口，添加了一些用于生命周期管理的方法（同时还有一些用于任务提交的便利方法）。在程序清单 6-7 中给出了 ExecutorService 中的生命周期管理方法。

程序清单 6-7　ExecutorService 中的生命周期管理方法

```
public interface ExecutorService extends Executor {
    void shutdown();
    List<Runnable> shutdownNow();
    boolean isShutdown();
    boolean isTerminated();
    boolean awaitTermination(long timeout, TimeUnit unit)
        throws InterruptedException;
    //  ……其他用于任务提交的便利方法
}
```

ExecutorService 的生命周期有 3 种状态：运行、关闭和已终止。ExecutorService 在初始创建时处于运行状态。shutdown 方法将执行平缓的关闭过程：不再接受新的任务，同时等待已经提交的任务执行完成——包括那些还未开始执行的任务。shutdownNow 方法将执行粗暴的关闭过程：它将尝试取消所有运行中的任务，并且不再启动队列中尚未开始执行的任务。

在 ExecutorService 关闭后提交的任务将由“拒绝执行处理器 (Rejected Execution Handler)”来处理（请参见 8.3.3 节），它会抛弃任务，或者使得 execute 方法抛出一个未检查的 RejectedExecutionException。等所有任务都完成后，ExecutorService 将转入终止状态。可以调用 awaitTermination 来等待 ExecutorService 到达终止状态，或者通过调用 isTerminated 来轮询 ExecutorService 是否已经终止。通常在调用 awaitTermination 之后会立即调用 shutdown，从而产生同步地关闭 ExecutorService 的效果。（第 7 章将进一步介绍 Executor 的关闭和任务取消等方面的内容。）

程序清单 6-8 的 LifecycleWebServer 通过增加生命周期支持来扩展 Web 服务器的功能。可以通过两种方法来关闭 Web 服务器：在程序中调用 stop，或者以客户端请求形式向 Web 服务器发送一个特定格式的 HTTP 请求。

程序清单 6-8　支持关闭操作的 Web 服务器

```
class LifecycleWebServer {
    private final ExecutorService exec = ...;

    public void start() throws IOException {
        ServerSocket socket = new ServerSocket(80);
```

```
        while (!exec.isShutdown()) {
            try {
                final Socket conn = socket.accept();
                exec.execute(new Runnable() {
                    public void run() { handleRequest(conn); }
                });
            } catch (RejectedExecutionException e) {
                if (!exec.isShutdown())
                    log("task submission rejected", e);
            }
        }
    }

    public void stop() { exec.shutdown(); }

    void handleRequest(Socket connection) {
        Request req = readRequest(connection);
        if (isShutdownRequest(req))
            stop();
        else
            dispatchRequest(req);
    }
}
```

6.2.5 延迟任务与周期任务

Timer 类负责管理延迟任务（“在 100ms 后执行该任务”）以及周期任务（“每 10ms 执行一次该任务”）。然而，Timer 存在一些缺陷，因此应该考虑使用 ScheduledThreadPoolExecutor 来代替它[⊖]。可以通过 ScheduledThreadPoolExecutor 的构造函数或 newScheduledThreadPool 工厂方法来创建该类的对象。

Timer 在执行所有定时任务时只会创建一个线程。如果某个任务的执行时间过长，那么将破坏其他 TimerTask 的定时精确性。例如某个周期 TimerTask 需要每 10ms 执行一次，而另一个 TimerTask 需要执行 40ms，那么这个周期任务或者在 40ms 任务执行完成后快速连续地调用 4 次，或者彻底“丢失”4 次调用（取决于它是基于固定速率来调度还是基于固定延时来调度）。线程池能弥补这个缺陷，它可以提供多个线程来执行延时任务和周期任务。

Timer 的另一个问题是，如果 TimerTask 抛出了一个未检查的异常，那么 Timer 将表现出糟糕的行为。Timer 线程并不捕获异常，因此当 TimerTask 抛出未检查的异常时将终止定时线程。这种情况下，Timer 也不会恢复线程的执行，而是会错误地认为整个 Timer 都被取消了。因此，已经被调度但尚未执行的 TimerTask 将不会再执行，新的任务也不能被调度。（这个问题称之为“线程泄漏 [Thread Leakage]”，7.3 节将介绍该问题以及如何避免它。）

在程序清单 6-9 的 OutOfTime 中给出了 Timer 中为什么会出现这种问题，以及如何使得试图提交 TimerTask 的调用者也出现问题。你可能认为程序会运行 6 秒后退出，但

⊖ Timer 支持基于绝对时间而不是相对时间的调度机制，因此任务的执行对系统时钟变化很敏感，而 ScheduledThreadPoolExecutor 只支持基于相对时间的调度。

实际情况是运行1秒就结束了，并抛出了一个异常消息“Timer already cancelled”。ScheduledThreadPoolExecutor能正确处理这些表现出错误行为的任务。在Java 5.0或更高的JDK中，将很少使用Timer。

如果要构建自己的调度服务，那么可以使用DelayQueue，它实现了BlockingQueue，并为ScheduledThreadPoolExecutor提供调度功能。DelayQueue管理着一组Delayed对象。每个Delayed对象都有一个相应的延迟时间：在DelayQueue中，只有某个元素逾期后，才能从DelayQueue中执行take操作。从DelayQueue中返回的对象将根据它们的延迟时间进行排序。

6.3　找出可利用的并行性

Executor框架帮助指定执行策略，但如果要使用Executor，必须将任务表述为一个Runnable。在大多数服务器应用程序中都存在一个明显的任务边界：单个客户请求。但有时候，任务边界并非是显而易见的，例如在很多桌面应用程序中。即使是服务器应用程序，在单个客户请求中仍可能存在可发掘的并行性，例如数据库服务器。（请参见[CPJ 4.4.1.1]了解在选择任务边界时的各种权衡因素及相关讨论。）

程序清单6-9　错误的Timer行为

```
public class OutOfTime {
    public static void main(String[] args) throws Exception {
        Timer timer = new Timer();
        timer.schedule(new ThrowTask(), 1);
        SECONDS.sleep(1);
        timer.schedule(new ThrowTask(), 1);
        SECONDS.sleep(5);
    }

    static class ThrowTask extends TimerTask {
        public void run() { throw new RuntimeException(); }
    }
}
```

本节中我们将开发一些不同版本的组件，并且每个版本都实现了不同程度的并发性。该示例组件实现浏览器程序中的页面渲染（Page-Rendering）功能，它的作用是将HTML页面绘制到图像缓存中。为了简便，假设HTML页面只包含标签文本，以及预定义大小的图片和URL。

6.3.1　示例：串行的页面渲染器

最简单的方法就是对HTML文档进行串行处理。当遇到文本标签时，将其绘制到图像缓存中。当遇到图像引用时，先通过网络获取它，然后再将其绘制到图像缓存中。这很容易实现，程序只需将输入中的每个元素处理一次（甚至不需要缓存文档），但这种方法可能会令用户感到烦恼，他们必须等待很长时间，直到显示所有的文本。

另一种串行执行方法更好一些，它先绘制文本元素，同时为图像预留出矩形的占位空间，在处理完了第一遍文本后，程序再开始下载图像，并将它们绘制到相应的占位空间中。在程序

清单 6-10 的 SingleThreadRenderer 中给出了这种方法。

图像下载过程的大部分时间都是在等待 I/O 操作执行完成，在这期间 CPU 几乎不做任何工作。因此，这种串行执行方法没有充分地利用 CPU，使得用户在看到最终页面之前要等待过长的时间。通过将问题分解为多个独立的任务并发执行，能够获得更高的 CPU 利用率和响应灵敏度。

程序清单 6-10 串行地渲染页面元素

```
public class SingleThreadRenderer {
    void renderPage(CharSequence source) {
        renderText(source);
        List<ImageData> imageData = new ArrayList<ImageData>();
        for (ImageInfo imageInfo : scanForImageInfo(source))
            imageData.add(imageInfo.downloadImage());
        for (ImageData data : imageData)
            renderImage(data);
    }
}
```

6.3.2 携带结果的任务 Callable 与 Future

Executor 框架使用 Runnable 作为其基本的任务表示形式。Runnable 是一种有很大局限的抽象，虽然 run 能写入到日志文件或者将结果放入某个共享的数据结构，但它不能返回一个值或抛出一个受检查的异常。

许多任务实际上都是存在延迟的计算——执行数据库查询，从网络上获取资源，或者计算某个复杂的功能。对于这些任务，Callable 是一种更好的抽象：它认为主入口点（即 call）将返回一个值，并可能抛出一个异常。[⊖]在 Executor 中包含了一些辅助方法能将其他类型的任务封装为一个 Callable，例如 Runnable 和 java.security.PrivilegedAction。

Runnable 和 Callable 描述的都是抽象的计算任务。这些任务通常是有范围的，即都有一个明确的起始点，并且最终会结束。Executor 执行的任务有 4 个生命周期阶段：创建、提交、开始和完成。由于有些任务可能要执行很长的时间，因此通常希望能够取消这些任务。在 Executor 框架中，已提交但尚未开始的任务可以取消，但对于那些已经开始执行的任务，只有当它们能响应中断时，才能取消。取消一个已经完成的任务不会有任何影响。（第 7 章将进一步介绍取消操作。）

Future 表示一个任务的生命周期，并提供了相应的方法来判断是否已经完成或取消，以及获取任务的结果和取消任务等。在程序清单 6-11 中给出了 Callable 和 Future。在 Future 规范中包含的隐含意义是，任务的生命周期只能前进，不能后退，就像 ExecutorService 的生命周期一样。当某个任务完成后，它就永远停留在“完成”状态上。

get 方法的行为取决于任务的状态（尚未开始、正在运行、已完成）。如果任务已经完成，

⊖ 要使用 Callable 来表示无返回值的任务，可使用 Callable<Void>。

那么 get 会立即返回或者抛出一个 Exception，如果任务没有完成，那么 get 将阻塞并直到任务完成。如果任务抛出了异常，那么 get 将该异常封装为 ExecutionException 并重新抛出。如果任务被取消，那么 get 将抛出 CancellationException。如果 get 抛出了 ExecutionException，那么可以通过 getCause 来获得被封装的初始异常。

程序清单 6-11　Callable 与 Future 接口

```
public interface Callable<V> {
    V call() throws Exception;
}

public interface Future<V> {
    boolean cancel(boolean mayInterruptIfRunning);
    boolean isCancelled();
    boolean isDone();
    V get() throws InterruptedException, ExecutionException,
                   CancellationException;
    V get(long timeout, TimeUnit unit)
        throws InterruptedException, ExecutionException,
               CancellationException, TimeoutException;
}
```

可以通过许多种方法创建一个 Future 来描述任务。ExecutorService 中的所有 submit 方法都将返回一个 Future，从而将一个 Runnable 或 Callable 提交给 Executor，并得到一个 Future 用来获得任务的执行结果或者取消任务。还可以显式地为某个指定的 Runnable 或 Callable 实例化一个 FutureTask。（由于 FutureTask 实现了 Runnable，因此可以将它提交给 Executor 来执行，或者直接调用它的 run 方法。）

从 Java 6 开始，ExecutorService 实现可以改写 AbstractExecutorService 中的 newTaskFor 方法，从而根据已提交的 Runnable 或 Callable 来控制 Future 的实例化过程。在默认实现中仅创建了一个新的 FutureTask，如程序清单 6-12 所示。

程序清单 6-12　ThreadPoolExecutor 中 newTaskFor 的默认实现

```
protected <T> RunnableFuture<T> newTaskFor(Callable<T> task) {
    return new FutureTask<T>(task);
}
```

在将 Runnable 或 Callable 提交到 Executor 的过程中，包含了一个安全发布过程（请参见 3.5 节），即将 Runnable 或 Callable 从提交线程发布到最终执行任务的线程。类似地，在设置 Future 结果的过程中也包含了一个安全发布，即将这个结果从计算它的线程发布到任何通过 get 获得它的线程。

6.3.3　示例：使用 Future 实现页面渲染器

为了使页面渲染器实现更高的并发性，首先将渲染过程分解为两个任务，一个是渲染所有的文本，另一个是下载所有的图像。（因为其中一个任务是 CPU 密集型，而另一个任务是 I/O

密集型，因此这种方法即使在单 CPU 系统上也能提升性能。）

Callable 和 Future 有助于表示这些协同任务之间的交互。在程序清单 6-13 的 FutureRenderer 中创建了一个 Callable 来下载所有的图像，并将其提交到一个 ExecutorService。这将返回一个描述任务执行情况的 Future。当主任务需要图像时，它会等待 Future.get 的调用结果。如果幸运的话，当开始请求时所有图像就已经下载完成了，即使没有，至少图像的下载任务也已经提前开始了。

程序清单 6-13 使用 Future 等待图像下载

```
public class FutureRenderer {
    private final ExecutorService executor = ...;

    void renderPage(CharSequence source) {
        final List<ImageInfo> imageInfos = scanForImageInfo(source);
        Callable<List<ImageData>> task =
                new Callable<List<ImageData>>() {
                    public List<ImageData> call() {
                        List<ImageData> result
                                = new ArrayList<ImageData>();
                        for (ImageInfo imageInfo : imageInfos)
                            result.add(imageInfo.downloadImage());
                        return result;
                    }
                };

        Future<List<ImageData>> future =  executor.submit(task);
        renderText(source);

        try {
            List<ImageData> imageData =  future.get();
            for (ImageData data : imageData)
                renderImage(data);
        } catch (InterruptedException e) {
            // 重新设置线程的中断状态
            Thread.currentThread().interrupt();
            // 由于不需要结果，因此取消任务
            future.cancel(true);
        } catch (ExecutionException e) {
            throw launderThrowable(e.getCause());
        }
    }
}
```

get 方法拥有“状态依赖”的内在特性，因而调用者不需要知道任务的状态，此外在任务提交和获得结果中包含的安全发布属性也确保了这个方法是线程安全的。Future.get 的异常处理代码将处理两个可能的问题：任务遇到一个 Exception，或者调用 get 的线程在获得结果之前被中断（请参见 5.5.2 节和 5.4 节）。

FutureRenderer 使得渲染文本任务与下载图像数据的任务并发地执行。当所有图像下载完后，会显示到页面上。这将提升用户体验，不仅使用户更快地看到结果，还有效利用了并行

性，但我们还可以做得更好。用户不必等到所有的图像都下载完成，而希望看到每当下载完一幅图像时就立即显示出来。

6.3.4 在异构任务并行化中存在的局限

在上个示例中，我们尝试并行地执行两个不同类型的任务——下载图像与渲染页面。然而，通过对异构任务进行并行化来获得重大的性能提升是很困难的。

两个人可以很好地分担洗碗的工作：其中一个人负责清洗，而另一个人负责烘干。然而，要将不同类型的任务平均分配给每个工人却并不容易。当人数增加时，如何确保他们能帮忙而不是妨碍其他人工作，或者在重新分配工作时，并不是容易的事情。如果没有在相似的任务之间找出细粒度的并行性，那么这种方法带来的好处将减少。

当在多个工人之间分配异构的任务时，还有一个问题就是各个任务的大小可能完全不同。如果将两个任务 A 和 B 分配给两个工人，但 A 的执行时间是 B 的 10 倍，那么整个过程也只能加速 9%。最后，当在多个工人之间分解任务时，还需要一定的任务协调开销：为了使任务分解能提高性能，这种开销不能高于并行性实现的提升。

FutureRenderer 使用了两个任务，其中一个负责渲染文本，另一个负责下载图像。如果渲染文本的速度远远高于下载图像的速度（可能性很大），那么程序的最终性能与串行执行时的性能差别不大，而代码却变得更复杂了。当使用两个线程时，至多能将速度提高一倍。因此，虽然做了许多工作来并发执行异构任务以提高并发度，但从中获得的并发性却是十分有限的。（在 11.4.2 节和 11.4.3 节中的示例说明了同一个问题。）

只有当大量相互独立且同构的任务可以并发进行处理时，才能体现出将程序的工作负载分配到多个任务中带来的真正性能提升。

6.3.5 CompletionService:Executor 与 BlockingQueue

如果向 Executor 提交了一组计算任务，并且希望在计算完成后获得结果，那么可以保留与每个任务关联的 Future，然后反复使用 get 方法，同时将参数 timeout 指定为 0，从而通过轮询来判断任务是否完成。这种方法虽然可行，但却有些繁琐。幸运的是，还有一种更好的方法：完成服务 (CompletionService)。

CompletionService 将 Executor 和 BlockingQueue 的功能融合在一起。你可以将 Callable 任务提交给它来执行，然后使用类似于队列操作的 take 和 poll 等方法来获得已完成的结果，而这些结果会在完成时将被封装为 Future。ExecutorCompletionService 实现了 CompletionService，并将计算部分委托给一个 Executor。

ExecutorCompletionService 的实现非常简单。在构造函数中创建一个 BlockingQueue 来保存计算完成的结果。当计算完成时，调用 Future-Task 中的 done 方法。当提交某个任务时，该任务将首先包装为一个 QueueingFuture，这是 FutureTask 的一个子类，然后再改写子类的 done 方法，并将结果放入 BlockingQueue 中，如程序清单 6-14 所示。take 和 poll 方法委托给了 BlockingQueue，这些方法会在得出结果之前阻塞。

程序清单 6-14 由 ExecutorCompletionService 使用的 QueueingFuture 类

```
private class QueueingFuture<V> extends FutureTask<V> {
    QueueingFuture(Callable<V> c) { super(c); }
    QueueingFuture(Runnable t, V r) { super(t, r); }

    protected void done() {
        completionQueue.add(this);
    }
}
```

6.3.6 示例：使用 CompletionService 实现页面渲染器

可以通过 CompletionService 从两个方面来提高页面渲染器的性能：缩短总运行时间以及提高响应性。为每一幅图像的下载都创建一个独立任务，并在线程池中执行它们，从而将串行的下载过程转换为并行的过程：这将减少下载所有图像的总时间。此外，通过从 CompletionService 中获取结果以及使每张图片在下载完成后立刻显示出来，能使用户获得一个更加动态和更高响应性的用户界面。如程序清单 6-15 的 Renderer 所示。

程序清单 6-15 使用 CompletionService，使页面元素在下载完成后立即显示出来

```
public class Renderer {
    private final ExecutorService executor;

    Renderer(ExecutorService executor) { this.executor = executor; }

    void renderPage(CharSequence source) {
        List<ImageInfo> info = scanForImageInfo(source);
        CompletionService<ImageData> completionService =
            new ExecutorCompletionService<ImageData>(executor);
        for (final ImageInfo imageInfo : info)
            completionService.submit(new Callable<ImageData>() {
                public ImageData call() {
                    return imageInfo.downloadImage();
                }
            });

        renderText(source);

        try {
            for (int t = 0, n = info.size(); t < n;  t++) {
                Future<ImageData> f = completionService.take();
                ImageData imageData = f.get();
                renderImage(imageData);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } catch (ExecutionException e) {
            throw launderThrowable(e.getCause());
        }
    }
}
```

多个 ExecutorCompletionService 可以共享一个 Executor，因此可以创建一个对于特定计算私有，又能共享一个公共 Executor 的 ExecutorCompletionService。因此，CompletionService 的作用就相当于一组计算的句柄，这与 Future 作为单个计算的句柄是非常类似的。通过记录提交给 CompletionService 的任务数量，并计算出已经获得的已完成结果的数量，即使使用一个共享的 Executor，也能知道已经获得了所有任务结果的时间。

6.3.7　为任务设置时限

有时候，如果某个任务无法在指定时间内完成，那么将不再需要它的结果，此时可以放弃这个任务。例如，某个 Web 应用程序从外部的广告服务器上获取广告信息，但如果该应用程序在两秒钟内得不到响应，那么将显示一个默认的广告，这样即使不能获得广告信息，也不会降低站点的响应性能。类似地，一个门户网站可以从多个数据源并行地获取数据，但可能只会在指定的时间内等待数据，如果超出了等待时间，那么只显示已经获得的数据。

在有限时间内执行任务的主要困难在于，要确保得到答案的时间不会超过限定的时间，或者在限定的时间内无法获得答案。在支持时间限制的 Future.get 中支持这种需求：当结果可用时，它将立即返回，如果在指定时限内没有计算出结果，那么将抛出 TimeoutException。

在使用限时任务时需要注意，当这些任务超时后应该立即停止，从而避免为继续计算一个不再使用的结果而浪费计算资源。要实现这个功能，可以由任务本身来管理它的限定时间，并且在超时后中止执行或取消任务。此时可再次使用 Future，如果一个限时的 get 方法抛出了 TimeoutException，那么可以通过 Future 来取消任务。如果编写的任务是可取消的（参见第 7 章），那么可以提前中止它，以免消耗过多的资源。在程序清单 6-13 和 6-16 的代码中使用了这项技术。

程序清单 6-16 给出了限时 Future.get 的一种典型应用。在它生成的页面中包括响应用户请求的内容以及从广告服务器上获得的广告。它将获取广告的任务提交给一个 Executor，然后计算剩余的文本页面内容，最后等待广告信息，直到超出指定的时间[⊖]。如果 get 超时，那么将取消[⊖]广告获取任务，并转而使用默认的广告信息。

程序清单 6-16　在指定时间内获取广告信息

```
Page renderPageWithAd() throws InterruptedException {
    long endNanos = System.nanoTime() + TIME_BUDGET;
    Future<Ad> f = exec.submit(new FetchAdTask());
    // 在等待广告的同时显示页面
    Page page = renderPageBody();
    Ad ad;
    try {
        // 只等待指定的时间长度
        long timeLeft = endNanos - System.nanoTime();
```

⊖ 传递给 get 的 timeout 参数的计算方法是，将指定时限减去当前时间。这可能会得到负数，但 java.util.concurrent 中所有与时限相关的方法都将负数视为零，因此不需要额外的代码来处理这种情况。

⊖ Future.cancel 的参数为 true 表示任务线程可以在运行过程中中断。请参见第 7 章。

```
        ad = f.get(timeLeft, NANOSECONDS);
    } catch (ExecutionException e) {
        ad = DEFAULT_AD;
    } catch (TimeoutException e) {
        ad = DEFAULT_AD;
        f.cancel(true);
    }
    page.setAd(ad);
    return page;
}
```

6.3.8 示例：旅行预定门户网站

“预定时间”方法可以很容易地扩展到任意数量的任务上。考虑这样一个旅行预定门户网站：用户输入旅行的日期和其他要求，门户网站获取并显示来自多条航线、旅店或汽车租赁公司的报价。在获取不同公司报价的过程中，可能会调用 Web 服务、访问数据库、执行一个 EDI 事务或其他机制。在这种情况下，不宜让页面的响应时间受限于最慢的响应时间，而应该只显示在指定时间内收到的信息。对于没有及时响应的服务提供者，页面可以忽略它们，或者显示一个提示信息，例如“Did not hear from Air Java in time。”

从一个公司获得报价的过程与从其他公司获得报价的过程无关，因此可以将获取报价的过程当成一个任务，从而使获得报价的过程能并发执行。创建 n 个任务，将其提交到一个线程池，保留 n 个 Future，并使用限时的 get 方法通过 Future 串行地获取每一个结果，这一切都很简单，但还有一个更简单的方法——invokeAll。

程序清单 6-17 使用了支持限时的 invokeAll，将多个任务提交到一个 ExecutorService 并获得结果。InvokeAll 方法的参数为一组任务，并返回一组 Future。这两个集合有着相同的结构。invokeAll 按照任务集合中迭代器的顺序将所有的 Future 添加到返回的集合中，从而使调用者能将各个 Future 与其表示的 Callable 关联起来。当所有任务都执行完毕时，或者调用线程被中断时，又或者超过指定时限时，invokeAll 将返回。当超过指定时限后，任何还未完成的任务都会取消。当 invokeAll 返回后，每个任务要么正常地完成，要么被取消，而客户端代码可以调用 get 或 isCancelled 来判断究竟是何种情况。

程序清单 6-17 在预定时间内请求旅游报价

```
private class QuoteTask implements Callable<TravelQuote> {
    private final TravelCompany company;
    private final TravelInfo travelInfo;
    ...
    public TravelQuote call() throws Exception {
        return company.solicitQuote(travelInfo);
    }
}

public List<TravelQuote> getRankedTravelQuotes(
        TravelInfo travelInfo, Set<TravelCompany> companies,
        Comparator<TravelQuote> ranking, long time, TimeUnit unit)
        throws InterruptedException {
```

```
    List<QuoteTask> tasks = new ArrayList<QuoteTask>();
    for (TravelCompany company : companies)
        tasks.add(new QuoteTask(company, travelInfo));

    List<Future<TravelQuote>> futures =
        exec.invokeAll(tasks, time, unit);

    List<TravelQuote> quotes =
        new ArrayList<TravelQuote>(tasks.size());
    Iterator<QuoteTask> taskIter = tasks.iterator();
    for (Future<TravelQuote> f : futures) {
        QuoteTask task = taskIter.next();
        try {
            quotes.add(f.get());
        } catch (ExecutionException e) {
            quotes.add(task.getFailureQuote(e.getCause()));
        } catch (CancellationException e) {
            quotes.add(task.getTimeoutQuote(e));
        }
    }

    Collections.sort(quotes, ranking);
    return quotes;
}
```

小结

通过围绕任务执行来设计应用程序，可以简化开发过程，并有助于实现并发。Executor 框架将任务提交与执行策略解耦开来，同时还支持多种不同类型的执行策略。当需要创建线程来执行任务时，可以考虑使用 Executor。要想在将应用程序分解为不同的任务时获得最大的好处，必须定义清晰的任务边界。某些应用程序中存在着比较明显的任务边界，而在其他一些程序中则需要进一步分析才能揭示出粒度更细的并行性。